Project Schedule Analysis

1. Task dependencies
Sprint 4 work spans therapist-flow improvements, admin portal stabilization (booking operations and dashboard controls), patient commerce (assistive device in cart with delivery date), and patient portal capabilities (secure account settings and therapist profile viewing). Some items have a strict logical order; others can proceed in parallel across AZ, AY, JH, RL, YL.

Therapist flow → patient-visible therapist information

- SCRUM-50 (Enhance Therapist Flow) hardens the therapist-side workflow and keeps therapist-related data and behavior consistent. It is the practical prerequisite for presenting accurate, up-to-date therapist information anywhere in the product.
- SCRUM-54 (Patient Portal — Therapist profile: view / up-to-date information) depends on therapist records and APIs being correct, current, and correctly authorized for patient read access. If SCRUM-50 is unstable or still changing mid-sprint, SCRUM-54 will churn (fields, contracts, permissions).

Patient portal — account security & lifecycle

- SCRUM-53 (Patient Portal — Secure patient profile access) implements secure account management for patients: password change, security questions, account deletion, and two-factor authentication (2FA). The intent is to protect sensitive health data if credentials are compromised and to give patients predictable recovery and closure paths.
- SCRUM-53 can run in parallel with SCRUM-54 once the team agrees on shared patient-auth/session rules (tokens, re-authentication for sensitive actions, logout semantics). End-to-end verification still couples them because both ship in the same patient portal and must not weaken each other's security posture.

Admin stabilization

- SCRUM-21 (Admin Portal — merge and stabilize booking operations, dashboard panel controls) lands a stable admin surface around booking and dashboard behavior. It reduces merge friction and regression risk for anyone depending on the same integration window, shared services, or deployment rhythm during the sprint—even when another story is not "admin UI" itself.

Patient commerce

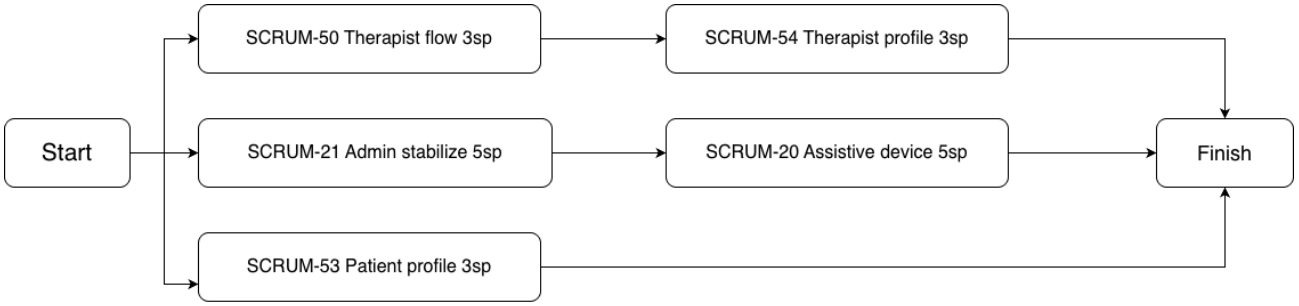
- SCRUM-20 (Add Assistive Device to Cart with Delivery Date) is a separate patient journey from the therapist-profile chain. It may reuse session/authentication and is integration-sensitive near the end of the sprint (cart rules, delivery date validation, API/UI alignment). It is primarily parallel in development to SCRUM-50 / SCRUM-53 / SCRUM-54, with a planned merge and test slot so commerce work is not stuck behind endless rebases.

Dependency chains (summary)
Start → SCRUM-50 → SCRUM-54
Start → SCRUM-53 (parallel with SCRUM-54 after shared patient-auth assumptions are set)

Start → SCRUM-21 (admin merge/stabilization track)
Start → SCRUM-20 (assistive-device cart + delivery date; parallel commerce track)

2. Network Diagram

The following simplified network diagram illustrates the logical sequence of development tasks during the sprint.



link

3. Critical path

The critical path is the dependent sequence that most constrains demo-ready end-to-end value and calendar time when work cannot be fully overlapped.

Primary dependency chain (patient trust — "what patients see about therapists")

- SCRUM-50 → SCRUM-54
  This is the longest strict sequence among patient-facing stories: therapist-side stability must exist before patient-facing therapist profiles can be finalized. Delay on SCRUM-50 directly delays SCRUM-54.

Calendar drivers (large, integration-heavy work)

- SCRUM-21 (5 SP) and SCRUM-20 (5 SP) are the largest stories. Even without a hard finish-to-start dependency between them, they often drive late merge risk, regression testing, and stabilization time. If either slips, the sprint still feels "blocked" at the system level because they touch core flows and shared codepaths.

Security-critical parallel chain

- SCRUM-53 is critical for trust (password/2FA/deletion/security questions). It may not sit on the same finish-to-start chain as SCRUM-50 → SCRUM-54, but it is release-critical because security mistakes are high-severity in a health-data context.

Secondary notes

In practice, the sprint's critical path is often max(critical merge/stabilization work, SCRUM-50→SCRUM-54 completion, SCRUM-53 hardening), because integration and QA cannot always run fully in parallel without contention.

4. Risk areas

- 50 + 54: Therapist data/API changes late → rework; bad patient-read permissions → privacy risk.
- 53: Auth flows (2FA, password, recovery, deletion) — edge-case bugs are high impact for health data.
- 53 + 54: Therapist profile must follow the same session / re-auth rules as account security.
- 21 / 20: Large merges + cart/date logic — late integration → regressions or last-minute fixes.
- Team parallelism: Long-lived branches → API drift and painful merges.

- Calendar: Course load can push integration and Jira updates to the end of the sprint.
- Mitigations: Early API contracts for 54; quick threat sketch for 53; one mid-sprint merge/test window; smoke tests after merges; clear DoD on 5-point stories.

5. Lessons learned

- Making SCRUM-50 → SCRUM-54 explicit during planning clarifies why therapist work cannot afford to be “finished in a bubble” without repeatedly invalidating patient profile work.
- SCRUM-53 should be sliced where possible (password change + tests before stacking 2FA + deletion) so security work produces incremental, verifiable progress instead of one risky end-of-sprint burst.
- Five-point stories (21, 20) benefit from early merge rehearsal and decomposition (stabilize vs feature vs polish) when the burndown shows long plateaus.
- Keeping SCRUM-20 conceptually separate from the therapist-profile chain prevents accidentally coupling commerce deadlines to therapist data readiness—unless a real dependency is discovered and then added to the diagram.